

Final Project vPlan: Configurable Packet Modifier (CPM)

Google Reichman – Chip Design & Verification

Author: Meitar Shimoni

Lecturer: Nadav Talmon

Date: 17/02/2025

Contents

Project Overview & Scope	2
DUT Summary	2
Scope	2
Requirements Extraction & RTM	3
Testbench Architecture	4
Verification Component Roles	4
Stimulus & Test Strategy	5
Mandatory UVM Mechanisms	5
Test Scenarios	5
Coverage & Closure Plan	5
Functional Coverage Targets	5
Sign-off Criteria	5

Project Overview & Scope

DUT Summary

The **Configurable Packet Modifier (CPM)** is a hardware block that processes single-beat packets received on a streaming input interface and produces transformed packets on a streaming output interface.

The DUT supports:

- Mode-dependent transformation (PASS, XOR, ADD, ROT)
- Optional packet drop based on opcode
- Ready/Valid flow control
- Deterministic, bounded latency
- Internal packet counters for observability

Latency is deterministic and mode-dependent:

Mode	Operation	Latency
PASS	$\text{payload_out} = \text{payload_in}$	0 cycles
XOR	$\text{payload_in} \wedge \text{MASK}$	1 cycles
ADD	$\text{payload_in} + \text{ADD_CONST}$	2 cycles
ROT	$\text{rotate_left}(\text{payload_in}, \text{ROT_AMT})$	1 cycles

Latency is defined from in_fire to out_fire.

Scope

In-Scope:

- Single-beat processing.
- Register configuration (RAL).
- Deterministic latency (0-2 cycles).
- Backpressure handling,
- Counter verification.

Out-of-Scope:

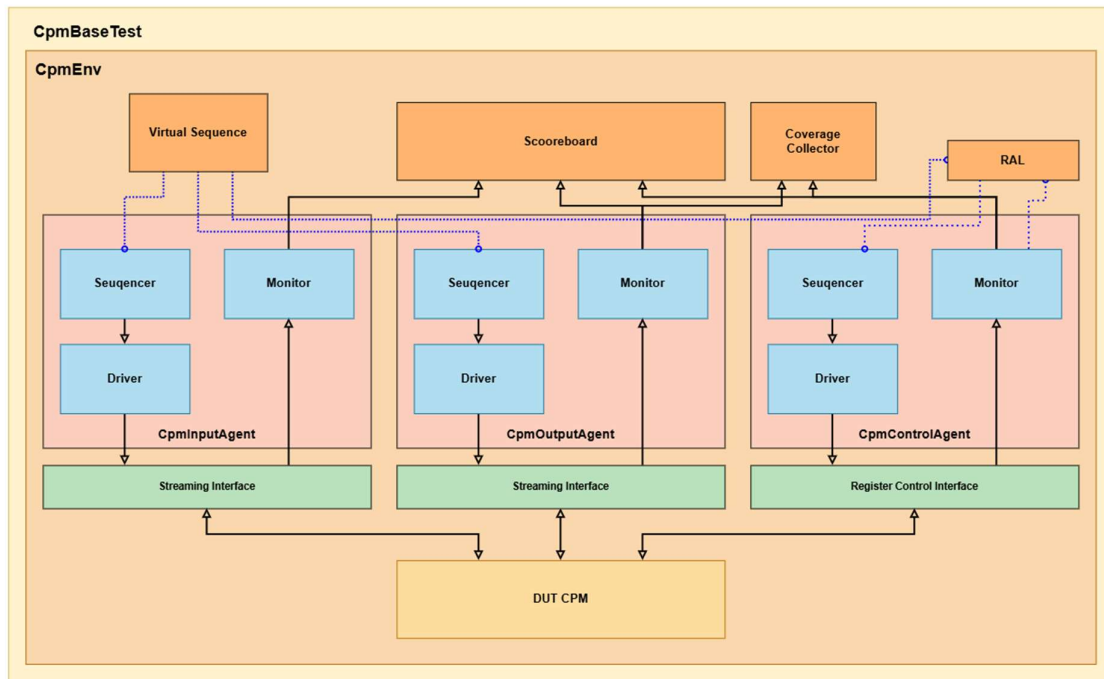
- Multi-beat packets.
- Reordering.
- Interrupts.
- DMA interfaces.

Requirements Extraction & RTM

Each requirement is verified using one or more of: Scoreboard comparison, SystemVerilog Assertions (SVA), directed tests, constrained-random testing, and RAL sequences.

Req ID	Description	Verification method	Planned Artifact
R1	Synchronous Reset: Upon rst, registers return to reset values, pipeline flushes, and counters clear.	Simulation / RAL	ResetSequence
R2	In-Order Processing: The CPM must preserve strict packet ordering under all conditions	Scoreboard	CpmScoreboard
R3	Handshake Stability (In): If in_valid is high and in_ready is low, data signals must remain stable	SVA	SVA inside StreamInInterface
R4	Handshake Stability (Out): If out_valid is high and out_ready is low, data signals must remain stable	SVA	SVA inside StreamOutInterface
R5	Processing Mode 0 (PASS): payload_out = payload_in with 0 cycles latency	Scoreboard / Test	SmokeTest
R6	Processing Mode 1 (XOR): payload_out = payload_in ^ MASK with 1 cycle latency.	Scoreboard / Test	SmokeTest
R7	Processing Mode 2 (ADD): payload_out = payload_in + ADD_CONST with 2 cycles latency	Scoreboard / Test	SmokeTest
R8	Processing Mode 3 (ROT): payload_out = rotate_left(payload_in, ROT_AMT) with 1 cycle latency.	Scoreboard / Test	SmokeTest
R9	Configuration Sampling: Mode and parameters are sampled at in_fire.	Scoreboard / Test	SmokeTest
R10	Packet Drop: Packets matching DROP_OPCODE when DROP_EN=1 are not produced at output.	Scoreboard	SmokeTest
R11	Counter Invariant: COUNT_OUT + DROPPED_COUNT == COUNT_IN.	Scoreboard	CpmScoreboard
R12	Backpressure: CPM must handle downstream stalls by deasserting in_ready.	Stress Test	Stress Test
R13	Bounded Liveness: Accepted non-dropped packets must exit within spec-defined latency.	SVA	SVA inside StreamInInterface
R14	Register Handshake: gnt is asserted whenever req is asserted (zero wait states).	SVA	SVA inside RegCtrlInterface

Testbench Architecture



Verification Component Roles

MyInputAgent: Drives packet id, opcode, and payload onto the streaming input.

MyOutputAgent: Manages out_ready to simulate downstream backpressure.

MyControlAgent: Handles register R/W using the req/gnt handshake.

RAL (Register Model):

- Mandatory access layer for all register interactions
- Uses `uvm_reg_adapter` and `uvm_reg_predictor`
- No direct bus driving allowed

Scoreboard & Reference Model

At every in_fire:

- Sample configuration registers
- Compute expected transformation
- Assign expected latency
- Evaluate drop decision
- Push expected transaction into FIFO (if not dropped)

At every out_fire:

- Compare output against FIFO head
- Enforce strict ordering
- Verify payload correctness
- Check latency bound

Functional coverage collector: Holds the cover groups for the whole environment.

Virtual sequence: Runs different sequences.

Stimulus & Test Strategy

Mandatory UVM Mechanisms

Control Model: Virtual sequences orchestrate scenarios; Tests use uvm_config_db for configuration.

RAL: Mandatory register model for all access (no direct bus driving) .

Factory Override: Use one override (e.g., base_traffic_seq -> coverage_traffic_seq) to hit rare opcode/mode crosses .

Callbacks: Implement a driver or monitor callback for event tagging or transaction modification.

Scoreboard: Maintains an expected queue; verifies the invariant: COUNT_OUT + DROPPED_COUNT == COUNT_IN.

Test Scenarios

Smoke Test: Basic Reset, RAL check, and PASS, XOR, ADD, ROT modes with traffic.

Stress Test: Burst traffic with randomized out_ready stalls to verify backpressure.

Drop Test: Specifically targets opcodes matching the DROP_CFG register.

Reconfig Test: Changes MODE during traffic to verify sampling at in_fire.

Coverage & Closure Plan

Functional Coverage Targets

- **MODE: 100% (All 4 modes).**
- **OPCODE: 90% (Distribute across 4-bit range).**
- **Cross: 80% (MODE x OPCODE).**
- **Event Bins: Hit at least one "Drop" event and one "Stall" event .**

Sign-off Criteria

- **100% Test pass rate.**
- **0 Scoreboard mismatches.**
- **0 SVA failures.**
- **Functional coverage targets met.**
- **RAL reset sequence passed.**